

Foundations

This chapter of the guide introduces the basic concepts of GStreamer. Understanding these concepts will be important in reading any of the rest of this guide, all of them assume understanding of these basic concepts.

Elements

An *element* is the most important class of objects in GStreamer. You will usually create a chain of elements linked together and let data flow through this chain of elements. An element has one specific function, which can be the reading of data from a file, decoding of this data or outputting this data to your sound card (or anything else). By chaining together several such elements, you create a *pipeline* that can do a specific task, for example media playback or capture. GStreamer ships with a large collection of elements by default, making the development of a large variety of media applications possible. If needed, you can also write new elements. That topic is explained in greater detail in the *GStreamer Plugin Writer's Guide*.

Pads

Pads are an element's input and output, where you can connect other elements. They are used to negotiate links and data flow between elements in GStreamer. A pad can be viewed as a "plug" or "port" on an element where links may be made with other elements, and through which data can flow to or from those elements. Pads have specific data handling capabilities: a pad can restrict the type of data that flows through it. Links are only allowed between two pads when the allowed data types (capabilities) of the two pads are compatible. Data types are negotiated between pads using a process called *caps negotiation*. Data types are described by `GstCaps`.

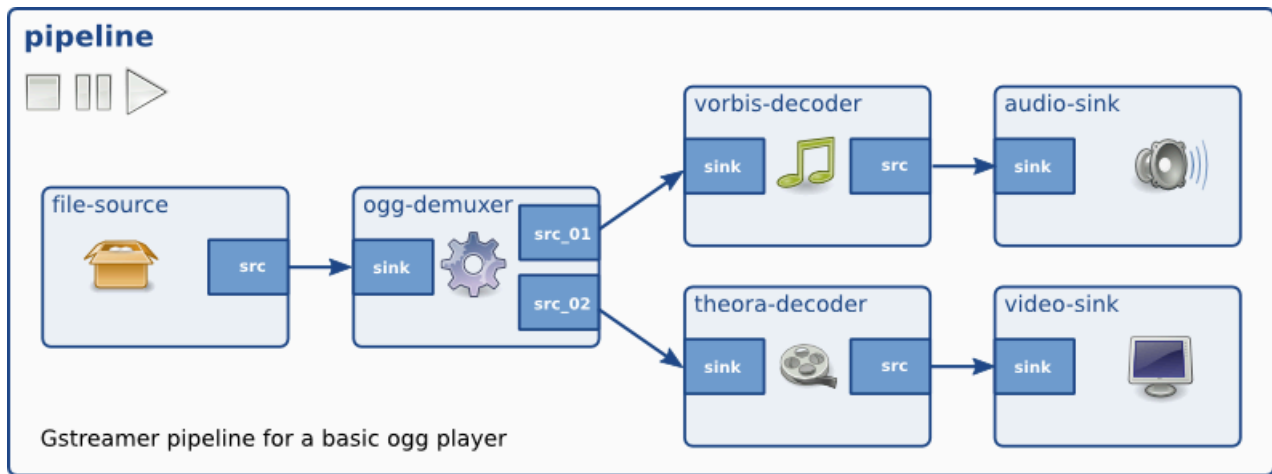
An analogy may be helpful here. A pad is similar to a plug or jack on a physical device. Consider, for example, a home theater system consisting of an audio amplifier, a DVD player, and a (silent) video projector. Linking the DVD player to the amplifier is allowed because both devices have audio jacks, and linking the projector to the DVD player is allowed because both devices have compatible video jacks. Links between the projector and the amplifier may not be made because the projector and amplifier have different types of jacks. Pads in GStreamer serve the same purpose as the jacks in the home theater system.

For the most part, all data in GStreamer flows one way through a link between elements. Data flows out of one element through one or more *source pads*, and elements accept incoming data through one or more *sink pads*. Source and sink elements have only source and sink pads, respectively. Data usually means buffers (described by the `GstBuffer` (<http://gstreamer.freedesktop.org/data/doc/gstreamer/stable/gstreamer/html/gstreamer-GstBuffer.html>) object) and events (described by the `GstEvent` (<http://gstreamer.freedesktop.org/data/doc/gstreamer/stable/gstreamer/html/gstreamer-GstEvent.html>) object).

Bins and pipelines

A *bin* is a container for a collection of elements. Since bins are subclasses of elements themselves, you can mostly control a bin as if it were an element, thereby abstracting away a lot of complexity for your application. You can, for example change state on all elements in a bin by changing the state of that bin itself. Bins also forward bus messages from their contained children (such as error messages, tag messages or EOS messages).

A *pipeline* is a top-level bin. It provides a bus for the application and manages the synchronization for its children. As you set it to `PAUSED` or `PLAYING` state, data flow will start and media processing will take place. Once started, pipelines will run in a separate thread until you stop them or the end of the data stream is reached.



Communication

GStreamer provides several mechanisms for communication and data exchange between the *application* and the *pipeline*.

- *buffers* are objects for passing streaming data between elements in the pipeline. Buffers always travel from sources to sinks (downstream).
- *events* are objects sent between elements or from the application to elements. Events can travel upstream and downstream. Downstream events can be synchronised to the data flow.
- *messages* are objects posted by elements on the pipeline's message bus, where they will be held for collection by the application. Messages can be intercepted synchronously from the streaming thread context of the element posting the message, but are usually handled asynchronously by the application from the application's main thread. Messages are used to transmit information such as errors, tags, state changes, buffering state, redirects etc. from elements to the application in a thread-safe way.
- *queries* allow applications to request information such as duration or current playback position from the pipeline. Queries are always answered synchronously. Elements can also use queries to request information from their peer elements (such as the file size or duration). They can be used both ways within a pipeline, but upstream queries are more common.

